



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE CIENCIAS

Tarea 3

ALUMNOS:

Alejandro Espino Gutiérrez

PROFESORES:

Javier León Cotonieto
Jose Manuel Evangelista Tiburcio
Adrián Martínez Manzo

ASIGNATURA

Sistemas Operativos

17 de septiembre de 2026

Vamos a tomar First-come-first-served y Priority Scheduling para implementar.

1 First-come-first-served

First-come-first-served es un algoritmo sencillo para la planificación de procesos. En general, lo que hace es que el primer proceso que llega a la cola es el primero en ser servido por el procesador, sin importar sus atributos como duración, prioridad, etc.

El funcionamiento del algoritmo es así:

- Un proceso p se marca como listo
- Se coloca p al final de la cola de listos
- Ejecutamos el primer proceso de la cola
- Cuando un proceso obtiene el procesador, no lo suelta hasta que termina su ráfaga de CPU, y no se interrumpe ese proceso para darle prioridad a cualquier otro proceso, sin importar si es urgente o más corto
- Cuando el proceso termina su ejecución, volvemos a obtener el primer proceso de la cola y ejecutarlo

La principal estructura de datos que se usa es una cola común FIFO con operaciones `enqueue` y `dequeue`

1.1 Implementación

Se implementó una función `void simulate_fcfs(Process *p, int n)`, en la cual primero se copió el arreglo original de procesos para no modificarlo en el main, y así no afectar al otro algoritmo a implementar, entonces luego se implementó un ordenamiento de los procesos por su orden de llegada (`arrival`) con insertion sort (por su simplicidad).

De ahí se entra en un ciclo en el cual primero el CPU se queda inactivo esperando a que llegue otro proceso (si no ha llegado), luego primero simulamos que el proceso se comienza a ejecutar con el reloj (calculando tiempos de espera y respuesta), luego avanzamos el reloj sumando el burst del proceso, calculamos tiempo de finalización y turnaround y finalmente actualizamos las métricas totales e imprimimos la fila nueva de la tabla.

Compilamos y ejecutamos la simulación con nuestro archivo `Procesos.txt` y obtenemos lo siguiente:

Simulación de FCFS								
P	arrival	burst	priority	start	finish	waiting	turnaround	response
P 1	0	2	1	0	2	0	2	0
P 2	2	4	2	2	6	0	4	0
P 3	4	6	3	6	12	2	8	2
P 4	6	8	4	12	20	6	14	6
P 5	8	10	5	20	30	12	22	12
P 6	10	2	1	30	32	20	22	20
P 7	12	4	2	32	36	20	24	20
P 8	14	6	3	36	42	22	28	22
P 9	16	8	4	42	50	26	34	26
P10	18	10	5	50	60	32	42	32
P11	20	2	1	60	62	40	42	40
P12	22	4	2	62	66	40	44	40
P13	24	6	3	66	72	42	48	42
P14	26	8	4	72	80	46	54	46
P15	28	10	5	80	90	52	62	52
P16	30	2	1	90	92	60	62	60
P17	32	4	2	92	96	60	64	60
P18	34	6	3	96	102	62	68	62
P19	36	8	4	102	110	66	74	66
P20	38	10	5	110	120	72	82	72
P21	40	2	1	120	122	80	82	80
P22	42	4	2	122	126	80	84	80
P23	44	6	3	126	132	82	88	82
P24	46	8	4	132	140	86	94	86
P25	48	10	5	140	150	92	102	92
P26	50	2	1	150	152	100	102	100
P27	52	4	2	152	156	100	104	100
P28	54	6	3	156	162	102	108	102
P29	56	8	4	162	170	106	114	106
P30	58	10	5	170	180	112	122	112
P31	60	2	1	180	182	120	122	120
P32	62	4	2	182	186	120	124	120
P33	64	6	3	186	192	122	128	122

P34	66	8	4	192	200	126	134	126
P35	68	10	5	200	210	132	142	132
P36	70	2	1	210	212	140	142	140
P37	72	4	2	212	216	140	144	140
P38	74	6	3	216	222	142	148	142
P39	76	8	4	222	230	146	154	146
P40	78	10	5	230	240	152	162	152
P41	80	2	1	240	242	160	162	160
P42	82	4	2	242	246	160	164	160
P43	84	6	3	246	252	162	168	162
P44	86	8	4	252	260	166	174	166
P45	88	10	5	260	270	172	182	172
P46	90	2	1	270	272	180	182	180
P47	92	4	2	272	276	180	184	180
P48	94	6	3	276	282	182	188	182
P49	96	8	4	282	290	186	194	186
P50	98	10	5	290	300	192	202	192
P51	100	2	1	300	302	200	202	200
P52	102	4	2	302	306	200	204	200
P53	104	6	3	306	312	202	208	202
P54	106	8	4	312	320	206	214	206
P55	108	10	5	320	330	212	222	212
P56	110	2	1	330	332	220	222	220
P57	112	4	2	332	336	220	224	220
P58	114	6	3	336	342	222	228	222
P59	116	8	4	342	350	226	234	226
P60	118	10	5	350	360	232	242	232
P61	120	2	1	360	362	240	242	240
P62	122	4	2	362	366	240	244	240
P63	124	6	3	366	372	242	248	242
P64	126	8	4	372	380	246	254	246
P65	128	10	5	380	390	252	262	252
P66	130	2	1	390	392	260	262	260
P67	132	4	2	392	396	260	264	260

P68	134	6	3	396	402	262	268	262
P69	136	8	4	402	410	266	274	266
P70	138	10	5	410	420	272	282	272
P71	140	2	1	420	422	280	282	280
P72	142	4	2	422	426	280	284	280
P73	144	6	3	426	432	282	288	282
P74	146	8	4	432	440	286	294	286
P75	148	10	5	440	450	292	302	292
P76	150	2	1	450	452	300	302	300
P77	152	4	2	452	456	300	304	300
P78	154	6	3	456	462	302	308	302
P79	156	8	4	462	470	306	314	306
P80	158	10	5	470	480	312	322	312
P81	160	2	1	480	482	320	322	320
P82	162	4	2	482	486	320	324	320
P83	164	6	3	486	492	322	328	322
P84	166	8	4	492	500	326	334	326
P85	168	10	5	500	510	332	342	332
P86	170	2	1	510	512	340	342	340
P87	172	4	2	512	516	340	344	340
P88	174	6	3	516	522	342	348	342
P89	176	8	4	522	530	346	354	346
P90	178	10	5	530	540	352	362	352
P91	180	2	1	540	542	360	362	360
P92	182	4	2	542	546	360	364	360
P93	184	6	3	546	552	362	368	362
P94	186	8	4	552	560	366	374	366
P95	188	10	5	560	570	372	382	372
P96	190	2	1	570	572	380	382	380
P97	192	4	2	572	576	380	384	380
P98	194	6	3	576	582	382	388	382
P99	196	8	4	582	590	386	394	386
P100	198	10	5	590	600	392	402	392
Promedios:								
waiting=194.00								
turnaround=200.00								
response=194.00								

2 Priority Scheduling

Priority Scheduling es un algoritmo de planificación de procesos donde cada proceso tiene una asignación de prioridad, y el CPU siempre asigna al proceso con prioridad más alta de entre los que están listos para ejecutarse, sin importar otros atributos como duración o el tiempo que lleve esperando (aunque en implementaciones más sofisticadas incrementa la prioridad de un proceso en función del tiempo que lleve esperando, pero para fines de nuestra implementación no). También nosotros por practicidad vamos a implementar una variante donde no haya interrupción i.e. una vez que el CPU está ejecutando un proceso, si llega un proceso de mayor prioridad, no interrumpimos, aunque en implementaciones

más sofisticadas suele tener interrupción

El funcionamiento del algoritmo es así:

- Un proceso p se marca como listo
- Se inserta p en la lista de listos en función de su prioridad asignada
- Ejecutamos el proceso con mayor prioridad y no se interrumpe ese proceso para ejecutar otro aunque el otro tenga mayor prioridad. Si dos tienen la misma, usamos FCFS para desempatar.
- Una vez que p termina su ejecución, volvemos a obtener el de mayor prioridad y lo ejecutamos.

La principal estructura de datos que vamos a usar es un arreglo, el cual vamos a reordenar por prioridad cada que insertemos un nuevo proceso.

2.1 Implementación

Se implementó la función `void simulate_priority(Process *p, int n)`, donde primero copiamos el arreglo en función de la arrival de cada proceso, luego definimos un arreglo de procesos que va a ser el de los listos

De ahí entramos en el ciclo, donde primero insertamos los procesos con arrival menor o igual al reloj actual, luego buscamos la posición donde vamos a insertar el proceso, primero en función a su prioridad, y luego hacemos un desempate por arrival. Ahora tomamos el proceso de mayor prioridad y simulamos la ejecución con el reloj, sumando el burst del proceso y calculando tiempo de respuesta y espera, luego calculamos tiempo de finalización y turnaround y finalmente actualizamos las métricas totales e imprimimos la fila nueva en la tabla. Luego insertamos procesos que pudieron haber llegado en la simulación del proceso.

Compilamos y ejecutamos la simulación con nuestro archivo `Procesos.txt` y obtenemos lo siguiente:

Simulación Priority Scheduling								
P	arrival	burst	priority	start	finish	waiting	turnaround	response
P 1	0	2	1	0	2	0	2	0
P 2	2	4	2	2	6	0	4	0
P 3	4	6	3	6	12	2	8	2
P 6	10	2	1	12	14	2	4	2
P 7	12	4	2	14	18	2	6	2
P 8	14	6	3	18	24	4	10	4
P11	20	2	1	24	26	4	6	4
P12	22	4	2	26	30	4	8	4
P16	30	2	1	30	32	0	2	0
P17	32	4	2	32	36	0	4	0
P13	24	6	3	36	42	12	18	12
P21	40	2	1	42	44	2	4	2
P22	42	4	2	44	48	2	6	2
P18	34	6	3	48	54	14	20	14
P26	50	2	1	54	56	4	6	4
P27	52	4	2	56	60	4	8	4
P31	60	2	1	60	62	0	2	0
P32	62	4	2	62	66	0	4	0
P23	44	6	3	66	72	22	28	22
P36	70	2	1	72	74	2	4	2
P37	72	4	2	74	78	2	6	2
P28	54	6	3	78	84	24	30	24
P41	80	2	1	84	86	4	6	4
P42	82	4	2	86	90	4	8	4
P46	90	2	1	90	92	0	2	0
P47	92	4	2	92	96	0	4	0
P33	64	6	3	96	102	32	38	32
P51	100	2	1	102	104	2	4	2
P52	102	4	2	104	108	2	6	2
P38	74	6	3	108	114	34	40	34
P56	110	2	1	114	116	4	6	4
P57	112	4	2	116	120	4	8	4
P61	120	2	1	120	122	0	2	0
P62	122	4	2	122	126	0	4	0
P43	84	6	3	126	132	42	48	42
P66	130	2	1	132	134	2	4	2

P67	132	4	2	134	138	2	6	2
P48	94	6	3	138	144	44	50	44
P71	140	2	1	144	146	4	6	4
P72	142	4	2	146	150	4	8	4
P76	150	2	1	150	152	0	2	0
P77	152	4	2	152	156	0	4	0
P53	104	6	3	156	162	52	58	52
P81	160	2	1	162	164	2	4	2
P82	162	4	2	164	168	2	6	2
P58	114	6	3	168	174	54	60	54
P86	170	2	1	174	176	4	6	4
P87	172	4	2	176	180	4	8	4
P91	180	2	1	180	182	0	2	0
P92	182	4	2	182	186	0	4	0
P63	124	6	3	186	192	62	68	62
P96	190	2	1	192	194	2	4	2
P97	192	4	2	194	198	2	6	2
P68	134	6	3	198	204	64	70	64
P73	144	6	3	204	210	60	66	60
P78	154	6	3	210	216	56	62	56
P83	164	6	3	216	222	52	58	52
P88	174	6	3	222	228	48	54	48
P93	184	6	3	228	234	44	50	44
P98	194	6	3	234	240	40	46	40
P 4	6	8	4	240	248	234	242	234
P 9	16	8	4	248	256	232	240	232
P14	26	8	4	256	264	230	238	230
P19	36	8	4	264	272	228	236	228
P24	46	8	4	272	280	226	234	226
P29	56	8	4	280	288	224	232	224
P34	66	8	4	288	296	222	230	222
P39	76	8	4	296	304	220	228	220
P44	86	8	4	304	312	218	226	218
P49	96	8	4	312	320	216	224	216
P54	106	8	4	320	328	214	222	214
P59	116	8	4	328	336	212	220	212
P64	126	8	4	336	344	210	218	210
P69	136	8	4	344	352	208	216	208

P74	146	8	4	352	360	206	214	206
P79	156	8	4	360	368	204	212	204
P84	166	8	4	368	376	202	210	202
P89	176	8	4	376	384	200	208	200
P94	186	8	4	384	392	198	206	198
P99	196	8	4	392	400	196	204	196
P 5	8	10	5	400	410	392	402	392
P10	18	10	5	410	420	392	402	392
P15	28	10	5	420	430	392	402	392
P20	38	10	5	430	440	392	402	392
P25	48	10	5	440	450	392	402	392
P30	58	10	5	450	460	392	402	392
P35	68	10	5	460	470	392	402	392
P40	78	10	5	470	480	392	402	392
P45	88	10	5	480	490	392	402	392
P50	98	10	5	490	500	392	402	392
P55	108	10	5	500	510	392	402	392
P60	118	10	5	510	520	392	402	392
P65	128	10	5	520	530	392	402	392
P70	138	10	5	530	540	392	402	392
P75	148	10	5	540	550	392	402	392
P80	158	10	5	550	560	392	402	392
P85	168	10	5	560	570	392	402	392
P90	178	10	5	570	580	392	402	392
P95	188	10	5	580	590	392	402	392
P100	198	10	5	590	600	392	402	392
Promedios:								
waiting=129.78								
turnaround=135.78								
response=129.78								

3 Comparación y conclusiones

Al comparar, vemos que la tabla de FCFS está ordenada por número de procesos, pues es por arrival, mientras que la de Priority no tiene un orden específico en los números de procesos. En cuanto a métricas, en promedios parece que Priority Scheduling es mucho mejor que FCFS, pues tenemos esto:

FCFS:

```
waiting=194.00
turnaround=200.00
response=194.00
```

Priority Scheduling:

```
waiting=129.78
```

```
turnaround=135.78  
response=129.78
```

Pero si analizamos a detalle procesos individuales, por prioridad, vemos que existe starvation en algunos, pues por ejemplo el proceso 1 fue el primero en ejecutarse con prioridad 1, pero habiendo llegado poco después del 1, el proceso 5 con prioridad 5 se ejecutó después del proceso 99, lo cual en una implementación práctica con muchos procesos y donde los procesos no terminan de llegar (como en un SO normal) los procesos con baja prioridad nunca se ejecutarían, lo cual es lo mismo a decir que van a tener starvation.

A pesar de ésto, podríamos evitar la starvation aumentando la prioridad de un proceso en función del tiempo que pasa esperando.

En conclusión, vemos que en cuanto a promedios de las métricas, Priority Scheduling es bastante mejor que FCFS, pero FCFS es estable y garantiza que ningún proceso va a tener starvation, mientras que Priority como lo implementamos en esta tarea va a causar que varios procesos de baja prioridad tengan starvation, incluso si llegaron poco después del primero. Ahora bien, si aumentamos la prioridad de un proceso en base al tiempo que lleva esperando, tendríamos un Priority Scheduling que garantice que ningún proceso se quede esperando infinitamente y por tanto ahí sería definitivamente mejor que FCFS, pero ambos tienen sus usos, dependiendo de los requerimientos del sistema que vayamos a implementar, si estamos en un sistema donde las prioridades no importen mucho o simplemente queramos que no existan prioridades, no tendría mucho sentido implementar Priority, pero sería lógico implementar FCFS, ahora bien si queremos implementar un sistema donde las prioridades sean muy importantes, sería bueno implementar Priority, pero aumentando la prioridad en base a tiempo de espera para evitar starvation.

El código de la implementación está en:

<https://github.com/alejandro-espino-gut/tarea3-SO.git>